

JavaScript Promises – Practice Worksheet

Part A – Concept Check (Write the Answer)

1. What is a Promise in JavaScript?
2. What are the three possible states of a Promise?
3. What is the difference between `resolve()` and `reject()`?
4. What does the `.then()` method do?
5. What is the purpose of `.catch()`?
6. Can multiple `.then()` methods be chained? What is it called?
7. What is the advantage of using Promises over callbacks?
8. How does `async` and `await` simplify working with Promises?

Part B – Predict the Output

1

```
const promise = new Promise((resolve, reject) => {  
  console.log("A");  
  resolve("B");  
  console.log("C");  
});
```

```
promise.then(value => console.log(value));  
console.log("D");
```

Output: _____

```
2. new Promise((resolve, reject) => {  
  setTimeout(() => resolve("Task Done"), 2000);  
}).then(msg => console.log(msg));
```

```
console.log("After Promise");
```

Output order: _____

3.

```
const serverOnline = false;
```

```
const getData = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    if (serverOnline) resolve("Data received");  
    else reject("Server not reachable");  
  }, 1000);  
});
```

```
getData  
  .then(data => console.log(data))  
  .catch(error => console.log(error))  
  .finally(() => console.log("Request completed"));
```

Output: _____

4

```
function step1() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      console.log("Step 1 done");  
      resolve();  
    }, 1000);  
  });  
}
```

```
function step2() {  
  console.log("Step 2 done");  
}
```

```
step1().then(step2);
```

What is printed, and in what order? _____

Part C – Complete the Code

1 Fill in the blanks to create a working Promise:

```
const fetchData = new Promise((_____, _____) => {  
  let dataAvailable = true;  
  if (dataAvailable)  
    _____("Data fetched successfully");  
  else  
    _____("No data found");  
});
```

2 Add .catch() to handle failure:

```
const orderPizza = new Promise((resolve, reject) => {  
  const available = false;  
  if (available) resolve("Pizza delivered");  
  else reject("Out of stock");  
});
```

// Add your catch handler here

3. Write a Promise chain that prints:

Step 1 complete

Step 2 complete

All done

Part D – Async & Await Practice

1 Convert the following to use async/await:

```
function getUser() {  
  return new Promise(resolve => {  
    setTimeout(() => resolve("User data received"), 1000);  
  });  
}
```

```
});  
}
```

```
getUser().then(msg => console.log(msg));
```

Write an async function called `loadData()` that:

- waits for `fetch("https://jsonplaceholder.typicode.com/posts/1")`
- converts it to JSON
- prints the title

Part E – Case Study (Application)

Case Study: Online Shopping Order Flow

You are simulating an online order process:

1. `placeOrder()` – takes 1 sec, resolves “Order placed”
2. `processPayment()` – takes 2 sec, resolves “Payment successful”
3. `deliverOrder()` – takes 3 sec, resolves “Order delivered”

Write code using **Promises and chaining** to print each message in order.